

8.5.4 In-Place Selection

In-place, nondestructive, selection is conceptually simple, but it requires a lot of bookkeeping, and it is correspondingly slow. The general idea is to pick some number M of elements at random, to sort them, and then to make a pass through the array *counting* how many elements fall in each of the $M + 1$ intervals defined by these elements. The k th largest will fall in one such interval — call it the “live” interval. One then does a second round, first picking M random elements in the live interval, and then determining which of the new, finer, $M + 1$ intervals all presently live elements fall into. And so on, until the k th element is finally localized within a single array of size M , at which point direct selection is possible.

How shall we pick M ? The number of rounds, $\log_M N = \log_2 N / \log_2 M$, will be smaller if M is larger; but the work to locate each element among $M + 1$ subintervals will be larger, scaling as $\log_2 M$ for bisection, say. Each round requires looking at all N elements, if only to find those that are still alive, while the bisections are dominated by the N that occur in the first round. Minimizing $O(N \log_M N) + O(N \log_2 M)$ thus yields the result

$$M \sim 2\sqrt{\log_2 N} \quad (8.5.3)$$

The square root of the logarithm is so slowly varying that secondary considerations of machine timing become important. We use $M = 64$ as a convenient constant value.

Further discussion, and code, is in a Webnote [5].

CITED REFERENCES AND FURTHER READING:

- Sedgewick, R. 1998, *Algorithms in C*, 3rd ed. (Reading, MA: Addison-Wesley), pp. 126ff.[1]
 Knuth, D.E. 1997, *Sorting and Searching*, 3rd ed., vol. 3 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley).
 Chambers, J.M., James, D.A., Lambert, D., and Vander Wiel, S. 2006, “Monitoring Networked Applications with Incremental Quantiles,” *Statistical Science*, vol. 21.[2]
 Tieney, L. 1983, “A Space-efficient Recursive Procedure for Estimating a Quantile of an Unknown Distribution,” *SIAM Journal on Scientific and Statistical Computing*, vol. 4, pp. 706–711.[3]
 Liechty, J.C., Lin, D.K.J., and McDermott, J.P. 2003, “Single-Pass Low-Storage Arbitrary Quantile Estimation for Massive Datasets,” *Statistics and Computing*, vol. 13, pp. 91–100.[4]
 Numerical Recipes Software 2007, “Code Listing for Selip,” *Numerical Recipes Webnote No. 11*, at <http://www.nr.com/webnotes?11> [5]

8.6 Determination of Equivalence Classes

A number of techniques for sorting and searching relate to data structures whose details are beyond the scope of this book, for example, trees, linked lists, etc. These structures and their manipulations are the bread and butter of computer science, as distinct from numerical analysis, and there is no shortage of books on the subject.

In working with experimental data, we have found that one particular such manipulation, namely the determination of equivalence classes, arises sufficiently often to justify inclusion here.

The problem is this: There are N “elements” (or “data points” or whatever), numbered $0, \dots, N - 1$. You are given pairwise information about whether the elements are in the same *equivalence class* of “sameness,” by whatever criterion happens to be of interest. For example, you may have a list of facts like: “Element 3 and element 7 are in the same class; element 19 and element 4 are in the same class; element 7 and element 12 are in the same class, . . .” Alternatively, you may have a procedure, given the numbers of two elements j and k , for deciding whether they are in the same class or different classes. (Recall that an equivalence relation can be anything satisfying the *RST properties*: reflexive, symmetric, transitive. This is compatible with any intuitive definition of “sameness.”)

The desired output is an assignment to each of the N elements of an equivalence class number, such that two elements are in the same class if and only if they are assigned the same class number.

Efficient algorithms work like this: Let $F(j)$ be the class or “family” number of element j . Start off with each element in its own family, so that $F(j) = j$. The array $F(j)$ can be interpreted as a tree structure, where $F(j)$ denotes the parent of j . If we arrange for each family to be its own tree, disjoint from all the other “family trees,” then we can label each family (equivalence class) by its most senior great-great- . . . grandparent. The detailed topology of the tree doesn’t matter at all, as long as we graft each related element onto it *somewhere*.

Therefore, we process each elemental datum “ j is equivalent to k ” by (i) tracking j up to its highest ancestor; (ii) tracking k up to its highest ancestor; and (iii) giving j to k as a new parent, or vice versa (it makes no difference). After processing all the relations, we go through all the elements j and reset their $F(j)$ ’s to their highest possible ancestors, which then label the equivalence classes.

The following routine, based on Knuth [1], assumes that there are m elemental pieces of information, stored in two arrays of length m , `lista`, `listb`, the interpretation being that `lista[j]` and `listb[j]`, $j=0 \dots m-1$, are the numbers of two elements that (we are thus told) are related.

```
eclass.h void eclass(VecInt_0 &nf, VecInt_I &lista, VecInt_I &listb)
Given m equivalences between pairs of n individual elements in the form of the input arrays
lista[0..m-1] and listb[0..m-1], this routine returns in nf[0..n-1] the number of the
equivalence class of each of the n elements, integers between 0 and n-1 (not all such integers
used).
{
    Int l,k,j,n=nf.size(),m=lista.size();
    for (k=0;k<n;k++) nf[k]=k;           Initialize each element its own class.
    for (l=0;l<m;l++) {                 For each piece of input information...
        j=lista[l];
        while (nf[j] != j) j=nf[j];      Track first element up to its ancestor.
        k=listb[l];
        while (nf[k] != k) k=nf[k];      Track second element up to its ancestor.
        if (j != k) nf[j]=k;            If they are not already related, make them
                                         so.
    }
    for (j=0;j<n;j++)                   Final sweep up to highest ancestors.
        while (nf[j] != nf[nf[j]]) nf[j]=nf[nf[j]];
}
```

Alternatively, we may be able to construct a boolean function `equiv(j,k)` that returns a value `true` if elements j and k are related, or `false` if they are not. Then we want to loop over all pairs of elements to get the complete picture. D. Eardley has devised a clever way of doing this while simultaneously sweeping the tree up to high ancestors in a manner that keeps it current and obviates most of the final sweep phase:

```
eclass.h void eclazz(VecInt_0 &nf, Bool equiv(const Int, const Int))
Given a user-supplied boolean function equiv that tells whether a pair of elements, each in the
range 0..n-1, are related, return in nf[0..n-1] equivalence class numbers for each element.
{
    Int kk,jj,n=nf.size();
    nf[0]=0;
    for (jj=1;jj<n;jj++) {              Loop over first element of all pairs.
```

```
nf[jj]=jj;
for (kk=0;kk<jj;kk++) {           Loop over second element of all pairs.
    nf[kk]=nf[nf[kk]];           Sweep it up this much.
    if (equiv(jj+1, kk+1)) nf[nf[nf[kk]]]=jj;
    Good exercise for the reader to figure out why this much ancestry is necessary!
}
}
for (jj=0;jj<n;jj++) nf[jj]=nf[nf[jj]];    Only this much sweeping is needed
                                           finally.
}
```

CITED REFERENCES AND FURTHER READING:

- Knuth, D.E. 1997, *Fundamental Algorithms*, 3rd ed., vol. 1 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §2.3.3.[1]
- Sedgewick, R. 1998, *Algorithms in C*, 3rd ed. (Reading, MA: Addison-Wesley), Chapter 30.